CI DIGITAL

MPPT

PLANO DE VERIFICAÇÃO

Grupo:

- ENZO
- GABRIEL MARCARINI CARAMURU
- LUCAS

Data: 28 de Outubro de 2026

Rev.: 1.0

1. ***Document Versions***

Número da Versão	Data	Autor	Sumário de Mudanças
1.0	27/10/2026	Gabriel Marcarini Caramuru	Criação do arquivo

Obs.: Os planos de verificação podem ser observados neste arquivo, entretanto é melhor visualizá-los separadamente nas pastas do repositório, uma vez que foram originalmente feitos em arquivo .md.

SPI Controller Verification Test Plan

1. Overview and Scope

This document details the verification plan for the `spi_controller` RTL module. The Design Under Test (DUT) is a parameterized SPI Master module, intended as a communication submodule for an MPPT system.

The verification strategy focuses on RTL-level simulation to ensure functional correctness, data integrity, and strict adherence to protocol timing constraints. This includes validating all four standard SPI modes, customizable clock generation, and handshake signals.

2. Test Environment Strategy

The primary testbench strategy relies on a **Self-Checking Loopback** topology. By connecting the `SDO` (Serial Data Out) directly to the `SDI` (Serial Data In), the testbench can automate data integrity checks by comparing the transmitted parallel word (`SDO_DATA`) with the received parallel word (`SDI_DATA`).

Protocol timing checks will be validated using waveform analysis and procedural assertions within the testbench to prepare the design for accurate static timing analysis (STA) and gate-level synthesis.

3. Test Cases

3.1 Basic Initialization and Reset (Smoke Test)

Objective: Verify the module safely recovers from reset and maintains safe idle states.

- **Stimulus:** Assert `RST_N` (active low) asynchronously during active system clock (`CLK`).
- **Expected Result:**
  - All internal state machines immediately return to `ST_IDLE`.
  - Outputs `BUSY`, `DONE`, `SCLK`, and `SDO` drop to 0 (or `CPOL` equivalent for `SCLK`).
  - Chip Select `CS_N` drives high (inactive) within 1 system clock cycle.
  - Inputs are ignored while `RST_N` is low.

3.2 Clock Divider Verification (`CLK_DIV`)

Objective: Validate that the SPI serial clock frequency is correctly derived from the system clock based on the `CLK_DIV` parameter.

- **Stimulus:** Parameterize the DUT with `CLK_DIV` = 1, 2, and 5. Initiate a dummy transfer.
- **Expected Result:**
  - The measured period of `SCLK` exactly matches the formula: `T_sclk = 2 * CLK_DIV * T_clk`.
  - For `CLK_DIV = 1`, `SCLK` runs at half the system clock frequency.

3.3 SPI Protocol Modes Validation (`CPOL` & `CPHA`)

Objective: Ensure the module correctly implements all four standard SPI communication modes.

- **Stimulus:** Run sequential transfers with configurations:
  - Mode 0: `CPOL=0, CPHA=0`
  - Mode 1: `CPOL=0, CPHA=1`
  - Mode 2: `CPOL=1, CPHA=0`

    - Mode 3: `CPOL=1, CPHA=1`
- **Expected Result:**
    - **Idle State:** `SCLK` matches the value of `CPOL` when `BUSY` is low.
    - **Shift/Sample Edges:** Data on `SDO` shifts on the correct propagating edge, and `SDI` is sampled on the correct sampling edge, precisely determined by the `CPHA` setting.

3.4 Data Integrity and Loopback (Full-Duplex)

Objective: Verify that data parallel-to-serial and serial-to-parallel conversions occur flawlessly (MSB first).

- **Stimulus:**
    - Connect `SDO` to `SDI` in the testbench.
    - Parameterize `DATA_WIDTH` = 12.
    - Send varied patterns: `12ThA5A` (alternating), `12ThFFF` (all ones), `12Th000` (all zeros), `12Th001` (walking one).
- **Expected Result:**
    - For every initiated transfer, upon the assertion of the `DONE` signal, `SDI_DATA` must exactly equal `SDO_DATA`.
    - The first bit transmitted on `SDO` must be the MSB of `SDO_DATA`.

3.5 Handshake & Protocol Timing Constraints

Objective: Validate structural protocol timing, ensuring safety margins for external SPI peripherals.

- **Stimulus:** Initiate a standard 12-bit transfer and monitor the transaction boundaries.
- **Expected Result:**
    - **CS Setup:** `CS_N` must assert (go low) at least 1 full `SCLK` period before the first active edge of `SCLK`.
    - **CS Hold:** `CS_N` must remain asserted (low) for at least 1 full `SCLK` period after the final active edge of `SCLK`.
    - **Done Pulse:** `DONE` must pulse high for exactly 1 system clock (`CLK`) cycle upon completion of

`ST_IDLE_GAP`.

- **Inter-Transaction Gap:** If a new `START` is immediately requested, the module must ensure `CS_N` remains de-asserted (high) for a minimum of 2 `SCLK` periods before starting the next transaction.

Sample Formatter Verification Test Plan

1. Overview and Scope

This document details the verification plan for the `sample_formatter` RTL module. The Design Under Test (DUT) receives 16-bit SPI frames containing a 4-bit Channel ID and a 12-bit ADC sample. The module extracts both fields, zero-extends the ADC sample to 16 bits, routes the formatted sample to the appropriate output according to the Channel ID, and generates a one-clock-cycle `sample_valid` pulse indicating that a new formatted sample is available.

Input Signals: `clk` , `rst_n` , `spi_frame_rx` , `spi_data_valid`
Output Signals: `v_sample_out` , `i_sample_out` , `sample_valid`

2. Verification Strategy

The verification strategy is based on RTL simulation using a self-checking testbench combined with SystemVerilog Assertions (SVA). Functional correctness is verified by comparing the DUT outputs against a reference formatter model implemented in the testbench. Assertions are used to verify FSM behavior, reset behavior, output protocol, and timing of the `sample_valid` signal.

3. Test Cases

3.1 Reset and Inicialization

objective: Ensure that reset initializes correctly.

- **Stimulus:** Assert and de-assert `rst_n` (active-low).
- **Expected Result:**
- FSM returs to `STATE_IDLE` ;
- `v_sample_out = 16'h0000` ;
- `i_sample_out = 16'h0000` ;
- `sample_valid = 1'b0` ;
- `r_formatted_data = 16'h0000` ;
- `r_channel_id = 4'h0` ;
- `r_adc_sample = 12'h000` .

3.2 Voltage Frame Formatting

objective: Verify correct formatting of a voltage sample.

- **Stimulus:**
  - Assert **`spi_data_valid` ;**
  - Send a frame with:
- Channel ID = `4'h1` ;
- ADC sample = known value.
- **Expected Result:**
  - `v_sample_out` receives the formatted value;
  - `i_sample_out` remains unchanged;
  - `sample_valid` asserted for one cycle.

3.3 Current Frame Formatting

objective: Verify correct formatting of a current sample.

- **Stimulus:**
  - Assert `spi_data_valid` ;
  - Send a frame with:
    - Channel ID = `4'h2` ;
    - ADC sample = known value.
- **Expected Result:**
  - `i_sample_out` receives the formatted value;
  - `v_sample_out` remains unchanged;
  - `sample_valid` asserted for one cycle.

3.4 Zero Extension

objective: Verify proper zero-extension from 12 bits to 16 bits.

- **Stimulus:**
  - Send several 12-bit values.
- **Expected Result:**
  - Correct extension (upper four bits must always be zero).

3.5 Invalid Channel ID

objective: Verify behavior for unsupported channel IDs.

- **Stimulus:**
  - Send samples using Channel IDs different from `4'h1` and `4'h2` .

- **Expected Result:**
  - Neither output is modified;
  - `sample_valid = 1'b0 .`

3.6 Minimum ADC Value

objective: Verify formatting of the minimum input.

Stimulus:

- ADC sample = `12'h000 .`

- **Expected Result:**
  - Formatted output = `16'h0000 .`

3.7 Maximum ADC Value

objective: Verify formatting of the maximum input.

- **Stimulus:**
  - ADC sample = `12'hFFF .`
- **Expected Result:**
  - Formatted output = `16'h0FFF .`

3.8 Consecutive Frames

objective: Verify continuous operation.

- **Stimulus:**
  - Send multiple consecutive SPI frames alternating voltage and current channels.
- **Expected Result:**
  - No frame lost;

- Every frame updates the corresponding output;
- Every frame produces one `sample_valid` pulse.

3.9 Random Frames

objective: Verify overall functionality.

- **Stimulus:**
  - Generate random Channel IDs and random ADC values.
- **Expected Result:**
  - The formatted outputs match the reference model for every valid transaction.

3.10 `sample_valid` Timing

objective: Verify protocol timing.

- **Stimulus:**
  - Send several valid SPI frames.
- **Expected Result:**
  - `sample_valid` is asserted for exactly one clock cycle;
  - Pulse occurs only after processing reaches the output stage;
  - No false pulses occur.

3.11 FSM State Transition

objective: Verify correct FSM sequence.

- **Stimulus:**
  - Send one valid SPI transaction.
- **Expected Result:**
  - FSM follows: `STATE_IDLE` -> `STATE_EXTEND_ALIGN` -> `STATE_CH_SEPARATOR` ->

    `STATE_REG_OUTPUT` -> `STATE_IDLE` ;
  - No illegal transitions occur.

3.12 No Valid Input

objective: Verify idle behavior.

- **Stimulus:**
  - Keep `spi_data_valid = 0` .
- **Expected Result:**
  - FSM remains in `STATE_IDLE` ;
  - Outputs remain unchanged;
  - `sample_valid` stays low.

Signal Processing Verification Test Plan

1. Overview and Scope

This document details the verification plan for the `signal_processing` RTL module. The Design Under Test (DUT) is a signal processing module, its function is to receive voltage and current samples, apply a four-sample moving average filter, and provide filtered outputs with a valid pulse indicating that a new filtered value is available.

Input Signals: `clk`, `rst_n`, `v_sample_out`, `i_sample_out`, `sample_valid` **Output Signals:** `v_signal_out`, `i_signal_out`, `signal_valid`

2. Verification Strategy

The verification strategy is based on RTL simulation using a self-checking testbench combined with SystemVerilog Assertions (SVA). Functional correctness is verified by comparing DUT outputs against a reference moving-average model implemented inside the testbench.
Assertions are used to verify protocol timing, reset behavior and output validity.

3. Test Cases

3.1 Reset and Inicialization

objective: Ensure that the reset initializes correctly.

- **Stimulus:** Assert and de-assert `rst_n` (active-low).
- **Expected Result:**
  - FSM returs to `STATE_IDLE`;
  - `v_signal_out = 16Th0000`;

- `i_signal_out = 16Th0000`;
- `signal_valid = 1Tb0`;
- `v_sum = 18Th00000`;
- `i_sum = 18Th00000`;
- All positions of `v_reg` and `v_reg` goes to `16Th0000`.

3.2 Single Sample Processing

objective: Check the first sample.

- **Stimulus:**
  - Assert `sample_valid` (active-high);
  - Assign one sample to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - `v_signal_out = v_sample_out / 4`;
  - `i_signal_out = i_sample_out / 4`.

3.3 Pipeline Filling

objective: Check the pipeline filling.

- **Stimulus:**
  - Assert `sample_valid` (active-high);
  - Assign four samples to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - First sample:
    - `v_signal_out = v_sample_1 / 4`;
    - `i_signal_out = i_sample_1 / 4`;
  - Second sample:
    - `v_signal_out = (v_sample_1 + v_sample_2) / 4`;
    - `i_signal_out = (i_sample_1 + i_sample_2) / 4`;
  - Third sample:
    - `v_signal_out = (v_sample_1 + v_sample_2 + v_sample_3) / 4`;

        - `i_signal_out = (i_sample_1 + i_sample_2 + i_sample_3) / 4;`
    - Fourth sample:
        - `v_signal_out = (v_sample_1 + v_sample_2 + v_sample_3 + v_sample_4) / 4;`
        - `i_signal_out = (i_sample_1 + i_sample_2 + i_sample_3 + i_sample_4) / 4`.

3.4 Sliding Window Verification

objective: Check the disposal of the oldest sample.

- **Stimulus:**
    - Assign five samples to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
    - Discard the oldest sample, so that the fifth sample is:
        - `v_signal_out = (v_sample_2 + v_sample_3 + v_sample_4 + v_sample_5) / 4;`
        - `i_signal_out = (i_sample_2 + i_sample_3 + i_sample_4 + i_sample_5) / 4`.

3.5 Constant Inputs

objective: Check stability.

- **Stimulus:**
    - Assign four equal samples to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
    - The final result must be equal to the input:
        - `v_signal_out = v_sample_out;`
        - `i_signal_out = i_sample_out`.

3.6 Increasing Ramp

objective: Check the rising average.

- **Stimulus:**
  - Assign rising samples to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - The final result must increase and correspond to a known result.

3.7 Decreasing Ramp

objective: Check the declining average.

- **Stimulus:**
  - Assign declining samples to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - The final result must decrease and correspond to a known result.

3.8 Maximum Values

objective: Check overflow.

- **Stimulus:**
  - Assign maximum values to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - The final result must correspond to a maximum known value;
  - The internal sum must not overflow.

3.9 Minimum Values

objective: Check zeros.

- **Stimulus:**
  - Assign zero to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - The final result must be zero.

3.10 Random Samples

objective: Check reliability and functionality.

- **Stimulus:**
  - Assign random values to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - The result must be equal to the one calculated automatically.

3.11 `signal_valid` Timing

objective: Ensure protocol.

- **Stimulus:**
  - Assign values to `v_sample_out` and `i_sample_out`.
- **Expected Result:**
  - Single pulse;
  - Occurs after processing;
  - Never occurs spontaneously.

MPPT Core Test Plan

Assumptions (Input Constraints)

`valid_signal_is_single_cycle_pulse`

Description: Legal transactions present `valid_signal` as a one-cycle pulse so the DUT's rising-edge detector starts exactly one MPPT evaluation cycle. The RTL detects the valid pulse as `valid_pulse = valid_signal & ~valid_d`, where `valid_d` is registered on `clk`.
Signals: `valid_signal` **Notes**:

```
$rose(valid_signal) |-> ##1 !valid_signal;
```

`sample_inputs_stable_during_evaluation`

Description: A legal MPPT sample holds `V` and `I` stable long enough for the controller sequence to compare the current power and voltage against the stored previous sample. The RTL computes `P_new = V * I` combinationally and reuses it throughout the FSM evaluation sequence. **Signals**: `V`, `I`, `valid_signal` **Notes**: After a `valid_signal` pulse, hold `V` and `I` stable for the bounded evaluation window (approximately four cycles). This is an input-only constraint.

Interface Assertions

reset_forces_idle_outputs

Type: Assertion **Targets**: `Dc_out`, `valid_end_cycle` **Description**: Asserting active-high `rst` asynchronously clears the edge detector and returns the FSM to the `IDLE` state. Since `Dc_out` defaults to `2Tb10` outside the action states and `valid_end_cycle` is asserted only during the `END` state, reset should force the observable idle output pattern. **Stimulus / Antecedent**:

```
rst == 1Tb1
```

Expected Consequent:

```
Dc_out == 2Tb10
  valid_end_cycle == 1Tb0
```

Timing: Same cycle (asynchronous reset) and remains true while reset is asserted. **Auxiliary Logic**: None. **Success Criteria**: Reset always presents the idle/no-action output state and suppresses completion.

---

no_completion_without_accepted_valid

Type: Assertion **Targets**: `valid_end_cycle` **Description**: `valid_end_cycle` shall not assert unless a previously accepted `valid_signal` rising edge has started an MPPT evaluation. The DUT leaves `IDLE` only on `valid_pulse`. **Stimulus / Antecedent**: No pending transaction exists, and no accepted `valid_signal` rising edge has occurred during the bounded observation window. **Expected Consequent**:

```
valid_end_cycle == 1Tb0
```

Timing: Continuous outside the one-to-four-cycle transaction window. **Auxiliary Logic**: 1-bit pending flag and a 3-bit transaction-age counter

started by `$rose(valid_signal)` and cleared by `valid_end_cycle`. **Success Criteria**: The completion flag never asserts spuriously during idle operation.

---

valid_end_single_cycle_pulse

Type: Assertion **Targets**: `valid_end_cycle` **Description**: Completion is encoded by the one-cycle `END` state because `valid_end_cycle = (state == END)`. The FSM immediately transitions from `END` back to `IDLE`. **Stimulus / Antecedent**:

```
valid_end_cycle == 1Tb1
```

Expected Consequent:

```
##1 !valid_end_cycle
```

Timing: One cycle. **Auxiliary Logic**: None. **Success Criteria**: Completion is a one-cycle pulse rather than a level.

equal_power_completes_without_action

Type: Assertion **Targets**: `valid_end_cycle`, `Dc_out` **Description**: When a newly accepted sample has the same computed power as the stored previous power, the FSM transitions directly from `IDLE` to `END` without changing the duty command from its default value. This matches the Perturb and Observe (P&O) algorithm, where no perturbation is required when the measured power does not change. **Stimulus / Antecedent**: Accepted `valid_signal` pulse when the auxiliary shadow register satisfies `prev_power == V * I`. **Expected Consequent**: `valid_end_cycle` asserts during the bounded completion cycle and `Dc_out == 2Tb10`. **Timing**: One cycle after the accepted sample, using the monitor's registered transaction-age

counter. **Auxiliary Logic**: Shadow `prev_power`, updated when `valid_end_cycle` completes; pending flag and transaction-age counter. **Success Criteria**: Equal-power samples complete without issuing action codes `2Tb00` or `2Tb01`.

---

same_direction_reference_increase_code

Type: Assertion **Targets**: `Dc_out` **Description**: When power and voltage change in the same direction, the controller perturbs the reference voltage in the same direction. The RTL maps both `(curr_power > prev_power && V > prev_V)` and `(curr_power < prev_power && V < prev_V)` to the `V_ref_G` state, where `Dc_out == 2Tb00`. **Stimulus / Antecedent**: Accepted sample satisfying `(curr_power > prev_power && V > prev_V)` or `(curr_power < prev_power && V < prev_V)`. **Expected Consequent**: `Dc_out == 2Tb00` during the bounded action cycle before completion. **Timing**: Second cycle after the accepted sample; completion follows one cycle later. **Auxiliary Logic**: Shadow `prev_power`, shadow `prev_V`, and transaction-age counter. **Success Criteria**: Same-direction power and voltage changes always produce the reference-increase command.

---

same_direction_sequence_completes

Type: Assertion **Targets**: `valid_end_cycle` **Description**: After taking the same-direction branch, the FSM must progress through the action state and eventually reach `END`, asserting `valid_end_cycle`. **Stimulus / Antecedent**: Same antecedent as `same_direction_reference_increase_code`. **Expected Consequent**: `valid_end_cycle == 1Tb1`. **Timing**: Third cycle after the accepted sample. **Auxiliary Logic**: Reuse the shadow registers and transaction-age counter. **Success Criteria**: The same-direction branch cannot deadlock before completion.

---

opposite_direction_reference_decrease_c

Type: Assertion **Targets**: `Dc_out` **Description**: When power and voltage change in opposite directions, the controller perturbs the reference voltage in the opposite direction. The RTL maps `(curr_power > prev_power && V < prev_V)` and `(curr_power < prev_power && V > prev_V)` to the `V_ref_L` state, where `Dc_out == 2Tb01`. **Stimulus / Antecedent**: Accepted sample satisfying `(curr_power > prev_power && V < prev_V)` or `(curr_power < prev_power && V > prev_V)`. **Expected Consequent**: `Dc_out == 2Tb01` during the bounded action cycle before completion. **Timing**: Second cycle after the accepted sample; completion follows one cycle later. **Auxiliary Logic**: Shadow `prev_power`, shadow `prev_V`, and transaction-age counter. **Success Criteria**: Opposite-direction power and voltage changes always produce the reference-decrease command.

opposite_direction_sequence_completes

Type: Assertion **Targets**: `valid_end_cycle` **Description**: After taking the opposite-direction branch, the FSM must progress through the action state and eventually reach `END`, asserting `valid_end_cycle`.
Stimulus / Antecedent: Same antecedent as `opposite_direction_reference_decrease_code`. **Expected Consequent**: `valid_end_cycle == 1Tb1`. **Timing**: Third cycle after the accepted sample. **Auxiliary Logic**: Reuse the shadow registers and transaction-age counter. **Success Criteria**: The opposite-direction branch cannot deadlock before completion.

ambiguous_delta_keeps_default_command

Type: Assertion **Targets**: `Dc_out` **Description**: If power changes but voltage remains unchanged relative to the stored previous voltage, neither directional P&O branch condition is satisfied. The FSM therefore transitions from `P_DIFF` directly to `END` without entering either `V_ref_L` or `V_ref_G`, leaving the duty command at its default value. **Stimulus / Antecedent**: Accepted `valid_signal` pulse where `curr_power != prev_power` and `V == prev_V`. **Expected Consequent**: `Dc_out == 2Tb10` throughout the bounded transaction; no action code is emitted.
Timing: Bounded over the two-cycle `P_DIFF` → `END` path. **Auxiliary Logic**: Shadow `prev_power`, shadow `prev_V`, pending flag, and transaction-age counter. **Success Criteria**: An ambiguous voltage delta never generates a false reference increase or decrease command.

`ambiguous_delta_completes`

Type: Assertion **Targets**: `valid_end_cycle` **Description**: Even when the voltage comparison cannot select either directional branch, the FSM must still terminate by taking the default `P_DIFF` → `END` path and asserting `valid_end_cycle`. **Stimulus / Antecedent**: Same antecedent as `ambiguous_delta_keeps_default_command`. **Expected Consequent**: `valid_end_cycle == 1Tb1`. **Timing**: Second cycle after the accepted sample. **Auxiliary Logic**: Reuse the shadow registers and transaction-age counter from `ambiguous_delta_keeps_default_command`. **Success Criteria**: The controller always completes the transaction, even when the voltage comparison is ambiguous.

`back_to_back_samples_use_previous_compl`

Type: Assertion **Targets**: `Dc_out`, `valid_end_cycle` **Description**: A sample accepted after a completed transaction shall be compared against the voltage and power captured at the previous completed transaction. The RTL updates `prev_V` and `prev_power` only during the

`END` state, so each new transaction must use the most recently completed sample as its comparison baseline. **Stimulus / Antecedent**: Drive one legal transaction to completion. Then drive a second legal `valid_signal` pulse with `V` and `I` selected so that the relationship between the second sample and the first completed sample produces a known directional branch. **Expected Consequent**: The second transaction emits the command corresponding to the relationship with the first completed sample (`Dc_out == 2Tb00` for same-direction deltas or `Dc_out == 2Tb01` for opposite-direction deltas), followed by `valid_end_cycle == 1Tb1`. **Timing**: During the second transaction, the action code is produced on the second cycle after the accepted sample, and `valid_end_cycle` asserts on the third cycle. **Auxiliary Logic**: Shadow `prev_power` and `prev_V` updated only when `valid_end_cycle` is observed, a two-transaction phase flag, and a transaction-age counter. **Success Criteria**: Consecutive transactions always use the previous completed sample as the comparison baseline rather than reset values or the current sample itself.

Formal Testplan — pwm_generator

Throughout this plan:

4. `DW = width + 1` — datapath width of `D_in` and the internal phase counter (11 bits by default).
5. `PERIOD = 2**(width+1)` — PWM period in `clk` cycles (2048 by default) (source: pwm_generator.sv:8-17).
6. `MAX_D = {(width+1){1'b1}}` — the full-scale duty code (11'h7FF = 2047 by default) (source: pwm_generator.sv:10). Note `PERIOD == MAX_D + 1` .
7. `mon_phase` — the monitor-side reference phase model described in `## Notes` .

Clocking and reset convention (applies to every scenario below). All assertions and covers are evaluated on `posedge clk` , the DUT's single native clock, with `disable iff (rst)` . Reset is harness infrastructure: no scenario names `rst` in its antecedent, precondition, or consequent, and no scenario checks reset sequencing or reset release. Individual scenarios therefore state only their data-side conditions.

Assumptions (input constraints)

Every value of `D_in` is architecturally legal, including mid-period changes: `D_in` is an unregistered control input consumed directly by the combinational output stage, with no double-buffering or

period-boundary latch (source: pwm_generator.sv:4,19-20). A global `$stable(D_in)` assumption would therefore remove exactly the dynamic-update behavior the design is built to support, and would make `duty_update_takes_effect_combinationally` vacuous.

Scenarios that require a steady duty code express that requirement **locally in their own antecedent** (`$stable(D_in)` / a held-value precondition) rather than as a global environment constraint.

Interface / protocol assertions

8. **pwm_out_asserts_only_for_nonzero_duty**
    - **Type:** assertion
    - **Description:** The output must never drive high while the commanded duty is zero. With `D_in == 0` the comparison `counter < D_in` is false for every phase, and the full-scale override does not apply because `MAX_D != 0` , so a high output would be a spurious pulse (source: pwm_generator.sv:19-20). Intentionally redundant — see the redundancy note in `## Notes` .
    - **Stimulus / antecedent:** `pwm_out == 1'b1`
    - **Expected output / consequent:** `D_in != {DW{1'b0}}`
    - **Timing:** Same cycle (combinational output).
    - **Success criteria:** A zero duty command can never produce an output pulse of any width.

9. **pwm_out_deasserts_only_below_full_scale**

    - **Type:** assertion
    - **Description:** The full-scale override `(D_in == MAX_D) ? 1'b1 : …` sits at the head of the output ternary and takes priority over the counter comparison, so a low output is only legal when the duty code is below full scale (source: pwm_generator.sv:19). This catches an override that is mis-decoded or ordered after the comparator. Intentionally redundant — see the redundancy note in `## Notes` .
    - **Stimulus / antecedent:** `pwm_out == 1'b0`
    - **Expected output / consequent:** `D_in != MAX_D`
    - **Timing:** Same cycle (combinational output).
    - **Success criteria:** No low cycle is ever observed while the full-scale duty code is applied.

10. **nonzero_duty_eventually_produces_a_pulse**

    - **Type:** assertion
    - **Description:** The free-running counter wraps modulo `2**(width+1)` with no stall or terminal-count reload, so any non-zero duty code that is held stable must produce a rising output within one full period — the generator cannot deadlock low (source: pwm_generator.sv:16,19-20).
    - **Stimulus / antecedent:** `D_in != 0` held stable (`$stable(D_in)`). Purely data-side; reset is handled by `disable iff (rst)` .
    - **Expected output / consequent:** `pwm_out == 1'b1` is observed.
    - **Timing:** Bounded — within `PERIOD` cycles of the antecedent. The bound is architectural (one counter

wrap), not open-ended, so a bounded window is used rather than unbounded liveness.

- **Auxiliary logic (minimal):** 1-bit "duty held stable and non-zero" flag; the bounded window is expressed as `##[0:PERIOD]` with no explicit counter.

- **Success criteria:** A stable non-zero duty always yields at least one rising edge per period.

End-to-end assertions

11. **output_level_matches_phase_comparison**

    - **Type:** assertion
    - **Description:** The core functional contract of the generator: below full scale, the output level at every cycle is exactly the magnitude comparison of the current period phase against the commanded duty, `pwm_out = (counter < D_in)` (source: pwm_generator.sv:19-20). The phase is reconstructible from ports alone, so this is a port-level prediction, not a DUT tap.
    - **Stimulus / antecedent:** `D_in != MAX_D`
    - **Expected output / consequent:** `pwm_out == (mon_phase < D_in)`
    - **Timing:** Same cycle — the output is a continuous assignment with no register stage.
    - **Auxiliary logic (minimal):** `mon_phase` — one `DW` -bit free-running reference phase counter, incremented every `posedge clk` , asynchronously cleared by `rst` (mirrors the DUT time

base; `rst` is used here only as the counter's clear, never as a property antecedent).

- **Success criteria:** The output level agrees with the reference comparison on every cycle of every period.

12. **pwm_rises_at_period_wrap**

    - **Type:** assertion
    - **Description:** Each period begins with the high phase. When the counter wraps from all-ones back to zero, the comparison `0 < D_in` becomes true for any non-zero duty, so the output must rise on the wrap boundary with no missing or doubled cycle — this is the property that makes successive periods contiguous (source: pwm_generator.sv:16,19-20).
    - **Stimulus / antecedent:** `mon_phase == 0` and `D_in != 0` .
    - **Expected output / consequent:** `pwm_out == 1'b1`
    - **Timing:** Same cycle as the zero phase.
    - **Modelling complexity:** HIGH — shares the single `DW` -bit `mon_phase` counter with the other end-to-end scenarios; no additional state.
    - **Success criteria:** Every period starts high whenever a non-zero duty is commanded.

13. **pwm_falls_when_phase_reaches_duty**

    - **Type:** assertion
    - **Description:** The high phase ends exactly at the commanded count. Because the comparator is strict less-than, the output must be low on the first cycle where the phase equals the duty code, which is what makes the measured high time equal `D_in` cycles per period rather

than `D_in + 1` (source: pwm_generator.sv:20). Paired with `pwm_rises_at_period_wrap`, this pins the pulse-width contract without a separate run-length counter.

- **Stimulus / antecedent:** `mon_phase == D_in` and `D_in != MAX_D`.
- **Expected output / consequent:** `pwm_out == 1'b0`
- **Timing:** Same cycle as the matching phase.
- **Auxiliary logic (minimal):** reuses `mon_phase` (`DW`-bit reference phase counter).
- **Success criteria:** The pulse terminates at exactly the commanded count, giving a high time of `D_in` cycles.

14. **duty_update_takes_effect_combinationally**

- **Type:** assertion
- **Description:** `D_in` is unregistered and feeds the comparator directly, so a duty update takes effect in the same cycle it is applied, with no end-of-period latching and no pipeline latency. A mid-pulse decrease truncates the in-flight pulse and a mid-period increase re-asserts the output immediately (source: pwm_generator.sv:4,19-20). This is the control path an upstream MPPT controller relies on.
- **Stimulus / antecedent:** `$changed(D_in)`
- **Expected output / consequent:** `pwm_out` already reflects the **new** `D_in` in the same cycle

  — i.e. `pwm_out == ((D_in == MAX_D) || (mon_phase < D_in))`.
- **Timing:** Zero latency — same cycle as the `D_in` change.
- **Auxiliary logic (minimal):** reuses `mon_phase` (`DW`-bit reference phase counter).
- **Success criteria:** No duty update is deferred to a period

boundary or delayed by a register stage.

Corner-case assertions

15. **zero_duty_forces_constant_low**

    - **Description:** Corner case 1 — the zero duty code. The strict less-than comparator (rather than less-than-or-equal) guarantees that `D_in == 0` yields no pulse at all, so 0% duty is exactly representable and the generator has a true off state (source: pwm_generator.sv:19-20).
    - **Stimulus / antecedent:** `D_in == {DW{1'b0}}`
    - **Expected output / consequent:** `pwm_out == 1'b0`
    - **Timing:** Same cycle, for every phase of the period.
    - **Auxiliary logic (minimal):** none
    - **Success criteria:** Zero duty never produces even a one-cycle pulse.

16. **full_scale_duty_forces_constant_high**

    - **Type:** assertion
    - **Description:** Corner case 2 — the full-scale override. When `D_in == MAX_D` the dedicated equality compare bypasses the counter entirely and forces a constant-high output, producing a true 100% duty instead of the `MAX_D/PERIOD` duty the bare comparator would give. Specifically, the bare comparator `mon_phase < MAX_D` would go low for one cycle at the top phase (`mon_phase == MAX_D`); the override suppresses exactly that notch (source: pwm_generator.sv:10,19).

- **Stimulus / antecedent:** `D_in == MAX_D`
- **Expected output / consequent:** `pwm_out == 1'b1` — including at `mon_phase == MAX_D`, where the bare comparator would have produced a low cycle.
- **Timing:** Same cycle, for every phase including the counter wrap.
- **Auxiliary logic (minimal):** none (the consequent is a constant; `mon_phase` is referenced only to describe the suppressed notch, not required by the check).
- **Success criteria:** Zero low samples are observed across complete periods at full scale.

17. **minimum_pulse_is_single_cycle**

    17.1 **Type:** assertion

    17.2 **Description:** Corner case 3 — the smallest resolvable pulse. With `D_in == 1` the output must be high only while the phase is zero and low for the remaining `PERIOD-1` cycles, confirming a one-clock minimum pulse width with no dead-time or minimum-pulse filtering (source: pwm_generator.sv:19-20).

    17.3 **Stimulus / antecedent:** `D_in == 1`

    17.4 **Expected output / consequent:** `pwm_out == (mon_phase == 0)`

    17.5 **Timing:** Same cycle, evaluated across the period.

    17.6 **Auxiliary logic (minimal):** reuses `mon_phase` (`DW`-bit reference phase counter).

    17.7 **Success criteria:** Exactly one high cycle per period at the minimum non-zero duty code.

18. **near_full_scale_has_two_low_cycles**

    18.1 **Description:** Proves the saturation override does **not**

trigger one code early. At `D_in == MAX_D - 1` the override does not apply, so the plain strict-less-than comparator governs: the output is high for phases `0 .. MAX_D-2` (that is `MAX_D - 1` cycles) and low for **two** phases, `MAX_D - 1` and `MAX_D` , because the period is `PERIOD = MAX_D + 1` cycles (source: pwm_generator.sv:19-20). If the override fired one code early the output would instead be constantly high, which this assertion detects.

18.2 **Stimulus / antecedent:** `D_in == MAX_D - 1`

18.3 **Expected output / consequent:** `pwm_out == (mon_phase < (MAX_D - 1))` — i.e. low exactly at `mon_phase` ∈ `{MAX_D-1, MAX_D}` .

18.4 **Timing:** Same cycle, evaluated across the period.

18.5 **Auxiliary logic (minimal):** reuses `mon_phase` (`DW` -bit reference phase counter).

18.6 **Success criteria:** Exactly two low cycles per period immediately below full scale, confirming the override boundary is at `MAX_D` and not lower.

19. **cover_zero_duty_held_across_full_period**

19.1 **Type:** cover

19.2 **Description:** Witness that the 0% duty corner is reachable and sustained — `D_in == 0`

held across a complete period with the output continuously low.

19.3 **Signals:** `D_in` , `pwm_out`

20. **cover_full_scale_across_counter_wrap**

20.1 **Type:** cover

20.2 **Description:** Witness the saturation path across the

counter rollover — `D_in == MAX_D` held while `mon_phase` passes through the top phase `MAX_D` and wraps to 0, with `pwm_out` continuously high. This is precisely the notch the override suppresses.

20.3 **Signals:** `D_in` , `pwm_out`

21. **cover_minimum_width_pulse**

21.1 **Type:** cover

21.2 **Description:** Witness a single-cycle output pulse (`D_in == 1`), confirming the minimum pulse width is reachable.

21.3 **Signals:** `D_in` , `pwm_out`

22. **cover_near_full_scale_two_cycle_low_run**

22.1 **Type:** cover

22.2 **Description:** Witness the **two**-cycle low run at `D_in == MAX_D - 1` , spanning phases `MAX_D-1` and `MAX_D` . Together with `cover_full_scale_across_counter_wrap` this distinguishes the override boundary: one code below full scale the output dips low twice, at full scale it never dips.

22.3 **Signals:** `D_in` , `pwm_out`

23. **cover_period_wrap_with_active_duty**

23.1 **Type:** cover

23.2 **Description:** Corner case 4 — witness the counter rollover from all-ones to zero while a mid-scale duty is active, with the output rising on the wrap boundary.

23.3 **Signals:** `D_in` , `pwm_out`

24. **cover_mid_period_duty_change_truncates_pulse**

24.1 **Type:** cover

24.2 **Description:** Corner case 5 — witness `D_in` dropping below the current phase while the output is high, truncating the in-flight pulse in the same cycle.

24.3 **Signals:** `D_in` , `pwm_out`